
Menu-Driven Paragraph Text Processor in C

*Word analysis, space removal, title case conversion and word-frequency counting using
strings, pointers and user-defined functions*

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

1. Problem Statement

Write a menu-driven C program that reads two paragraphs and performs the operations listed below. Each operation must be written as a separate user-defined function, and call by reference (pointers / arrays) must be used wherever a function modifies a string or returns more than one result.

1. Read two paragraphs from the user.
2. Identify and display the longest and shortest word of each paragraph.
3. Remove repeated consecutive spaces from each paragraph and display the result.
4. Convert each paragraph to title case and display the result.
5. Find and display the frequency of every distinct word (case-insensitive) in each paragraph.
6. Exit.

2. Concepts Used

Concept	Where it is used
Strings (char arrays)	Paragraphs and words are stored as null-terminated character arrays.
Pointers / call by reference	Arrays are passed as the address of their first element, so a function can modify the caller's string or return several results (for example longest and shortest word) through its parameters.
User-defined functions	Every operation has its own function, plus displayMenu() for the menu.
strtok(), strlen(), strcpy(), strcmp()	Splitting a paragraph into words, finding word length, copying and comparing words.
isalpha(), isspace(), toupper(), tolower()	Character-by-character processing in space removal, title case and frequency counting.
do-while loop and switch	Menu loop that repeats until the user chooses Exit.

3. Program Design

3.1 Functions

Function	Prototype, purpose and working
readParagraph()	<pre>void readParagraph(char paragraph[], size_t size);</pre> Reads one line with <code>fgets()</code>, removes the trailing newline and discards any extra characters beyond the buffer size.
findLongestAndShortestWords()	<pre>void findLongestAndShortestWords(const char paragraph[], char longestWord[], char shortestWord[]);</pre> Copies the paragraph (<code>strtok()</code> modifies its input), splits it into words with <code>strtok()</code>, measures each with <code>strlen()</code> and stores the longest and shortest using <code>strcpy()</code>. Both results are returned through the output arrays.

Function	Prototype, purpose and working
removeRepeatedSpaces()	<pre>void removeRepeatedSpaces(const char paragraph[], char modifiedParagraph[]);</pre> Scans character by character using <code>isspace()</code>. Only the first character of every run of whitespace is copied to the output.
convertToTitleCase()	<pre>void convertToTitleCase(const char paragraph[], char titleCaseParagraph[]);</pre> Uses a flag <code>newWord</code>. The first letter of each word is converted with <code>toupper()</code> and the remaining letters with <code>tolower()</code>; <code>isalpha()</code> and <code>isspace()</code> decide what is a letter or a word break.
findWordFrequencies()	<pre>void findWordFrequencies(const char paragraph[]);</pre> Splits the paragraph with <code>strtok()</code>, converts each word to lower case with <code>tolower()</code>, searches the list of known words using <code>strcmp()</code>, then either increments its count or stores the new word with <code>strcpy()</code>. Prints the table.
displayMenu()	<pre>void displayMenu(void);</pre> Prints the menu of options.

3.2 Call by reference

In C, an array name passed to a function is converted to a pointer to its first element. The function therefore works on the caller's original memory, which is exactly call by reference. This is used in the program as follows:

- `findLongestAndShortestWords()` writes two results into the caller's arrays `longestWord` and `shortestWord`.
- `removeRepeatedSpaces()` and `convertToTitleCase()` write the modified paragraph into the caller's output array.
- `readParagraph()` fills the caller's paragraph array with the text typed by the user.
- `const` is used on input paragraphs to guarantee that those functions do not change the original text.

3.3 Program flow

1. Display the menu and read the user's choice.
2. If the choice is 2 to 5 and paragraphs have not been read yet, ask the user to choose option 1 first.
3. Option 1: read both paragraphs. Options 2 to 5: call the matching function for paragraph 1 and paragraph 2 and display the results.
4. Option 6: print a goodbye message and stop. Any other value is reported as an invalid choice.
5. Repeat from step 1 until the user chooses Exit.

3.4 Assumptions

- A paragraph is entered as a single line of up to 1000 characters.
- Words are separated by whitespace and the punctuation characters `. , ; : ! ? " ( ) [ ] { } -` so punctuation is not counted as part of a word.
- Ties for longest or shortest word go to the first such word; frequencies are listed in order of first appearance.

4. Source Code

paragraph_processor.c (compiled with gcc -Wall -Wextra -std=c99 -pedantic without any warnings)

```
/*
 * -----
 * Project : Menu-Driven Paragraph Text Processor
 * Name    : Rudra Pratap Singh
 * SAP ID  : 590032811
 * Batch   : 16
 *
 * Description:
 * Reads two paragraphs and performs the following operations using
 * separate user-defined functions (pointers / arrays are used so that
 * results are returned through the arguments - call by reference):
 * 1. Read two paragraphs
 * 2. Find the longest and shortest word of each paragraph
 * 3. Remove repeated consecutive spaces
 * 4. Convert to title case
 * 5. Find the frequency of every distinct word (case-insensitive)
 * 6. Exit
 * -----
 */

#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <stddef.h>

#define PARA_SIZE    1001 /* max characters in a paragraph + '\0' */
#define MAX_WORDS    600 /* max distinct words handled for frequency */
#define MAX_WORD_LEN  50  /* max length stored for a word (frequency) */
#define DELIMS        " \t\r\n.,;:!?\"'()[]{}-_" /* characters that separate words */

/* ----- Function prototypes ----- */
void readParagraph(char paragraph[], size_t size);
void findLongestAndShortestWords(const char paragraph[],
                                char longestWord[], char shortestWord[]);
void removeRepeatedSpaces(const char paragraph[], char modifiedParagraph[]);
void convertToTitleCase(const char paragraph[], char titleCaseParagraph[]);
void findWordFrequencies(const char paragraph[]);
void displayMenu(void);

/* ----- main ----- */
int main(void)
{
    char paragraphs[2][PARA_SIZE]; /* the two paragraphs */
    char longestWord[PARA_SIZE];    /* result: longest word */
    char shortestWord[PARA_SIZE];   /* result: shortest word */
    char result[PARA_SIZE];         /* result: modified paragraph */
    char line[32];                  /* buffer for reading menu choice */
    int choice = 0;
    int i;
    int paragraphsRead = 0;         /* flag: 1 once option 1 is completed */

    paragraphs[0][0] = paragraphs[1][0] = '\0';

    do
    {
        displayMenu();

        if (fgets(line, sizeof(line), stdin) == NULL)
            choice = 6; /* end of input -> exit */
        else if (sscanf(line, "%d", &choice) != 1)
            choice = 0; /* non-numeric -> invalid */

        if (choice >= 2 && choice <= 5 && !paragraphsRead)
        {
            printf("\nPlease read the paragraphs first (choose option 1).\n");
            continue;
        }

        switch (choice)
        {

```

```

        case 1:
            for (i = 0; i < 2; i++)
            {
                printf("\nEnter paragraph %d:\n", i + 1);
                readParagraph(paragraphs[i], sizeof(paragraphs[i]));
            }
            paragraphsRead = 1;
            printf("\nBoth paragraphs have been read successfully.\n");
            break;

        case 2:
            for (i = 0; i < 2; i++)
            {
                findLongestAndShortestWords(paragraphs[i],
                                            longestWord, shortestWord);
                printf("\nParagraph %d:\n", i + 1);
                if (longestWord[0] == '\0')
                    printf(" No words found.\n");
                else
                {
                    printf(" Longest word : %s (length %zu)\n",
                           longestWord, strlen(longestWord));
                    printf(" Shortest word : %s (length %zu)\n",
                           shortestWord, strlen(shortestWord));
                }
            }
            break;

        case 3:
            for (i = 0; i < 2; i++)
            {
                removeRepeatedSpaces(paragraphs[i], result);
                printf("\nParagraph %d after removing repeated spaces:\n%s\n",
                       i + 1, result);
            }
            break;

        case 4:
            for (i = 0; i < 2; i++)
            {
                convertToTitleCase(paragraphs[i], result);
                printf("\nParagraph %d in title case:\n%s\n", i + 1, result);
            }
            break;

        case 5:
            for (i = 0; i < 2; i++)
            {
                printf("\nWord frequencies in paragraph %d:\n", i + 1);
                findWordFrequencies(paragraphs[i]);
            }
            break;

        case 6:
            printf("\nExiting the program. Thank you!\n");
            break;

        default:
            printf("\nInvalid choice! Please enter a number from 1 to 6.\n");
    }
} while (choice != 6);

return 0;
}

/* ----- displayMenu ----- */
void displayMenu(void)
{
    printf("\n=====\n");
    printf("      PARAGRAPH TEXT PROCESSOR\n");
    printf("=====\n");
    printf(" 1. Read two paragraphs\n");
    printf(" 2. Find longest and shortest words\n");
    printf(" 3. Remove repeated consecutive spaces\n");
    printf(" 4. Convert to title case\n");
    printf(" 5. Find word frequencies\n");
    printf(" 6. Exit\n");
    printf("-----\n");
}

```

```

    printf("Enter your choice: ");
}

/* ----- readParagraph ----- */
/* Reads one line of text into 'paragraph' (at most size-1 characters). */
void readParagraph(char paragraph[], size_t size)
{
    size_t len;
    int ch;

    if (fgets(paragraph, (int)size, stdin) == NULL)
    {
        paragraph[0] = '\0';
        return;
    }

    len = strlen(paragraph);
    if (len > 0 && paragraph[len - 1] == '\n')
        paragraph[len - 1] = '\0';          /* remove the newline */
    else if (len == size - 1)
    {
        while ((ch = getchar()) != '\n' && ch != EOF)
            ;                                /* discard extra input */
        printf("Note: paragraph was longer than %zu characters and was truncated.\n",
               size - 1);
    }
}

/* ----- findLongestAndShortestWords ----- */
/* strtok() modifies its input, so the paragraph is copied first.
 * The results are returned through longestWord and shortestWord. */
void findLongestAndShortestWords(const char paragraph[],
                                char longestWord[], char shortestWord[])
{
    char temp[PARA_SIZE];
    char *token;
    size_t len, maxLen = 0, minLen = 0;
    int first = 1;

    longestWord[0] = '\0';
    shortestWord[0] = '\0';

    strcpy(temp, paragraph);
    token = strtok(temp, DELIMS);

    while (token != NULL)
    {
        len = strlen(token);

        if (first)
        {
            strcpy(longestWord, token);
            strcpy(shortestWord, token);
            maxLen = minLen = len;
            first = 0;
        }
        else
        {
            if (len > maxLen)
            {
                strcpy(longestWord, token);
                maxLen = len;
            }
            if (len < minLen)
            {
                strcpy(shortestWord, token);
                minLen = len;
            }
        }
        token = strtok(NULL, DELIMS);
    }
}

/* ----- removeRepeatedSpaces ----- */
/* Scans character by character; a run of whitespace is replaced by a
 * single space. */
void removeRepeatedSpaces(const char paragraph[], char modifiedParagraph[])

```

```

{
    size_t i, j = 0;
    int prevSpace = 0;

    for (i = 0; paragraph[i] != '\0'; i++)
    {
        if (isspace((unsigned char)paragraph[i]))
        {
            if (!prevSpace)
                modifiedParagraph[j++] = ' ';
            prevSpace = 1;
        }
        else
        {
            modifiedParagraph[j++] = paragraph[i];
            prevSpace = 0;
        }
    }
    modifiedParagraph[j] = '\0';
}

/* ----- convertToTitleCase ----- */
/* First letter of every word -> upper case, remaining letters -> lower. */
void convertToTitleCase(const char paragraph[], char titleCaseParagraph[])
{
    size_t i;
    int newWord = 1;          /* 1 when the next letter starts a word */

    for (i = 0; paragraph[i] != '\0'; i++)
    {
        unsigned char c = (unsigned char)paragraph[i];

        if (isalpha(c))
        {
            titleCaseParagraph[i] = (char)(newWord ? toupper(c) : tolower(c));
            newWord = 0;
        }
        else
        {
            titleCaseParagraph[i] = (char)c;
            if (isspace(c) || c == '-') /* hyphenated words: Well-Known */
                newWord = 1;
        }
    }
    titleCaseParagraph[i] = '\0';
}

/* ----- findWordFrequencies ----- */
/* Counts each distinct word ignoring letter case and prints the table
 * (words are listed in the order of their first appearance). */
void findWordFrequencies(const char paragraph[])
{
    static char words[MAX_WORDS][MAX_WORD_LEN]; /* distinct words */
    int count[MAX_WORDS];                       /* their frequencies */
    char temp[PARA_SIZE];
    char current[MAX_WORD_LEN];
    char *token;
    int total = 0, i, k, found;

    strcpy(temp, paragraph);
    token = strtok(temp, DELIMS);

    while (token != NULL)
    {
        /* copy the word in lower case (long words are cut to fit) */
        for (k = 0; token[k] != '\0' && k < MAX_WORD_LEN - 1; k++)
            current[k] = (char)tolower((unsigned char)token[k]);
        current[k] = '\0';

        /* search the word in the list of words found so far */
        found = 0;
        for (i = 0; i < total; i++)
        {
            if (strcmp(words[i], current) == 0)
            {
                count[i]++;
                found = 1;
                break;
            }
        }
        if (!found)
        {
            strcpy(words[total], current);
            count[total] = 1;
            total++;
        }
    }
}

```

```

    }
}

if (!found && total < MAX_WORDS)
{
    strcpy(words[total], current);
    count[total] = 1;
    total++;
}
token = strtok(NULL, DELIMS);
}

if (total == 0)
{
    printf(" No words found.\n");
    return;
}

printf(" %-20s %s\n", "Word", "Frequency");
printf(" %-20s %s\n", "-----", "-----");
for (i = 0; i < total; i++)
    printf(" %-20s %d\n", words[i], count[i]);
}

```

5. Sample Output

The output below was produced by actually running the program. It shows the error message for choosing an option before reading the paragraphs, all five operations on two paragraphs that contain extra spaces and mixed letter case, and an invalid menu choice (7). The menu is displayed again after every operation in the real program but is shown only once here to save space.

```
=====
PARAGRAPH TEXT PROCESSOR
=====
1. Read two paragraphs
2. Find longest and shortest words
3. Remove repeated consecutive spaces
4. Convert to title case
5. Find word frequencies
6. Exit
-----
Enter your choice: 2

Please read the paragraphs first (choose option 1).

Enter your choice: 1

Enter paragraph 1:
The quick brown fox jumps over the lazy dog. The DOG barks; the fox runs away!

Enter paragraph 2:
C programming is fun, and Programming in C is POWERFUL.

Both paragraphs have been read successfully.

Enter your choice: 2

Paragraph 1:
Longest word : quick (length 5)
Shortest word : The (length 3)

Paragraph 2:
Longest word : programming (length 11)
Shortest word : C (length 1)

Enter your choice: 3

Paragraph 1 after removing repeated spaces:
The quick brown fox jumps over the lazy dog. The DOG barks; the fox runs away!

Paragraph 2 after removing repeated spaces:
C programming is fun, and Programming in C is POWERFUL.

Enter your choice: 4

Paragraph 1 in title case:
The Quick Brown Fox Jumps Over The Lazy Dog. The Dog Barks; The Fox Runs Away!

Paragraph 2 in title case:
C Programming Is Fun, And Programming In C Is Powerful.

Enter your choice: 5

Word frequencies in paragraph 1:
Word Frequency
-----
the 4
quick 1
brown 1
fox 2
jumps 1
over 1
lazy 1
dog 2
```

```
barks      1
runs       1
away       1
```

Word frequencies in paragraph 2:

Word	Frequency
c	2
programming	2
is	2
fun	1
and	1
in	1
powerful	1

Enter your choice: 7

Invalid choice! Please enter a number from 1 to 6.

Enter your choice: 6

Exiting the program. Thank you!

6. Conclusion

The program meets all the requirements of the problem statement. Every task is implemented in its own user-defined function, results are passed back through array (pointer) parameters, and the standard library functions `strtok()`, `strlen()`, `strcpy()`, `strcmp()`, `isalpha()`, `isspace()`, `toupper()` and `tolower()` are used as suggested. Working on this project gave practice with strings, pointers, modular programming and menu-driven design in C.